Documentación Técnica y Revisión de Código: CIOdD-UCR PDF Intelligence Pipeline V5.2 (Research & RAG Edition)

1. Propósito y Resumen General

El **CIOdD-UCR PDF Intelligence Pipeline (Versión 5.2, Esquema 5.2.0)** es un sistema de ingesta, estructuración, auditoría e indexación documental diseñado en el Centro de Investigación Observatorio del Desarrollo (CIOdD) de la Universidad de Costa Rica (UCR). Su objetivo principal es transformar colecciones de documentos PDF heterogéneos (nativos digitales, escaneados o híbridos) en un **corpus estructurado en formatos JSON, JSONL, Parquet, DuckDB e índices vectoriales/léxicos** listos para ser consumidos por Modelos de Lenguaje Grande (LLMs), sistemas de Generación Aumentada por Recuperación (RAG) y flujos de investigación científica auditable.

Principios de Diseño en V5.2

- **Cero pérdida de texto (`chunk_coverage >= 0.995`)**: Ningún párrafo ni sección corta se descarta; las secciones menores a 60 palabras se fusionan de forma controlada con el fragmento adyacente o se emiten marcadas con `short_section = True`.
- **Trazabilidad bibliográfica exacta**: Separación estricta entre el texto propio de cada fragmento (`unique_words`) y la ventana de solapamiento (`overlap_words`), garantizando que `page_start` y `page_end` reflejen exclusivamente las páginas donde reside la evidencia primaria, junto con su etiqueta impresa (`page_label`).
- **Control de falsos positivos estructurales**: Sistema de puntuación multicriterio para encabezados (`HeadingDetector`) con degradación de cabeceras repetidas y clasificador heurístico de tablas (`AdaptiveTableExtractor`) que separa tablas genuinas de prosa fragmentada (`tables_rejected.jsonl`).
- **Auditoría e integridad automatizada**: Generación automática de huellas criptográficas SHA-256 de entradas y salidas (`manifest.json`), métricas desacopladas de fiabilidad técnica (`extraction_quality_score`) y verificación de 9 reglas de consistencia (`integrity_report.json`).

---

2. Arquitectura y Flujo de Procesamiento

El pipeline procesa un directorio de entrada (`/content/PDFs`) en 10 fases secuenciales por documento y 4 fases globales de consolidación del corpus en `/content/pdf_intelligence`:

Fase	Componente / Clase	Función Principal	Sa
1. Perfilado	`PDFProfiler`	Calcula `sha256` del archivo, extrae metadatos internos con `pypdf` y muestrea hasta 7 páginas con `pymupdf` para preclasificar el PDF (`digital`, `scanned`, `hybrid`, `empty`).	Di do (d
2. Layout Nativo	`LayoutExtractor`	Extrae con `page.get_text("dict", sort=True)` los bloques de texto (`type=0`), metadatos de imágenes (`type=1`), `spans` tipográficos (fuente, tamaño, `bbox`) y la mediana tipográfica de la página (`median_font_size`).	Bl
3. Detección de Encabezados (Paso 1)	`HeadingDetector`	Evalúa cada bloque con reglas de exclusión (notas al pie, viñetas, leyendas, números de eje) y suma señales tipográficas/léxicas (`score >= 2.0`) asignando `heading_confidence` y `heading_level` (1 a 3).	At he he he bl
4. OCR Adaptativo	`OCRExtractor`	Si el texto nativo tiene `< 80` caracteres, `< 20` palabras o `text_quality_score < 0.35`, renderiza la página a 200 DPI y ejecuta Tesseract (`spa+eng`). Sustituye al texto nativo si aporta `> 20%` más palabras o si el nativo tenía `< 20` palabras.	oc ex oc
5. Extracción y Filtrado de Tablas	`AdaptiveTableExtractor`	Ejecuta `pdfplumber` priorizando la estrategia `"lines"` y usando `"text"` solo como respaldo si `"lines"` no halla tablas válidas. Evalúa métricas de densidad y	ta ta ta (d

Fase	Componente / Clase	Función Principal	Sa
		fragmentación; genera versión `markdown`, `records` y `caption`.	
6. Calidad y Limpieza de Bordes	`QualityAssessor` + `infer_repeated_lines`	Calcula puntajes de calidad de página, detecta patrones repetidos en las 3 primeras y 3 últimas líneas en `>= 35%` de las páginas (normalizando dígitos a `#`) y los elimina solo en los bordes.	`cl` `re` `ex` `ov` `pa`
7. Degradación de Falsos Encabezados (Paso 2)	`demote_repeated_headings`	En documentos de `>= 8` páginas, degrada bloques marcados como `is_heading` que se repiten en `>= max(4, ceil(8% páginas))` (cabeceras corridas o etiquetas recurrentes).	Bl (h `"r`
8. Construcción de Secciones	`SectionBuilder`	Agrupa bloques en secciones lógicas (`{doc_id}_S0001`). Une encabezados consecutivos sin texto intermedio (`" "` si están a `< 24pt`, o `" › "` como ruta jerárquica). En páginas OCR utiliza directamente `clean_text`.	`se` `se`
9. Segmentación en Chunks	`ChunkBuilder`	Divide párrafos largos (`> 800` palabras) por oraciones, empaqueta texto hacia `CHUNK_TARGET_WORDS` (550 palabras, máx. 800), aplica solapamiento intrasección de 80 palabras y fusiona secciones cortas (`< 60` palabras) sin descartar texto.	`ch` `ch`
10. Consolidación, Índices y Auditoría	`CorpusBuilder`	Exporta JSONL/Parquet, crea la base relacional `corpus.duckdb`, entrena matriz `TF-IDF` y vectores densos `FAISS`, y ejecuta `check_integrity()`.	`co` `tf` `em` `ma` `in`

Fase	Componente / Clase	Función Principal	Sa
			qu da

---

3. Especificación de Archivos JSON / JSONL para Análisis con LLMs

El directorio `/content/pdf_intelligence` produce dos familias de archivos JSON:

- **Archivos `.jsonl` (JSON Lines)**: Un objeto JSON por línea, diseñados para ingesta en *streaming*, carga en bases vectoriales, *fine-tuning* o suministro de contexto a LLMs. Cada entidad principal cuenta además con su espejo tabular `.parquet` (donde las listas y diccionarios anidados se serializan como cadenas JSON mediante `flatten_row`).
- **Archivos `.json` de control y gobierno**: Documentos únicos con metadatos de auditoría, integridad y diccionario de datos.

3.1. `chunks.jsonl` (Unidad primaria para LLMs y RAG)

Es el archivo central para alimentar ventanas de contexto de LLMs. Cada registro representa un fragmento semántico autocontenido con trazabilidad completa:

Campo	Tipo	Descripción y Uso en LLMs / RAG
`chunk_id`	`string`	Identificador único secuencial por documento (`doc_<16hex>_C00001`). Usar como llave de citación en prompts.
`document_id`	`string`	Identificador único del PDF padre (`doc_<16hex>`).
`filename`	`string`	Nombre del archivo PDF original.
`section_id`	`string`	Identificador de la sección contenedora (`doc_<16hex>_S0001`).

Campo	Tipo	Descripción y Uso en LLMs / RAG
`section_title`	`string`	Título de la sección a la que pertenece el fragmento.
`section_title_source`	`string`	`"detected"` (encabezado detectado en layout) o `"none"` (`"Contenido inicial"`).
`section_title_reliable`	`bool`	`true` si `section_title_source == "detected"` y `heading_confidence >= 0.6`. Indica si es seguro anteponer el título al texto al indexar.
`merged_section_ids`	`list[str]`	Lista de `section_id` de secciones cortas (`< 60` palabras) absorbidas dentro de este chunk.
`short_section`	`bool`	`true` si el chunk corresponde a una sección corta que no pudo fusionarse con el chunk previo.
`text`	`string`	Texto limpio y normalizado del fragmento (incluye las palabras de solapamiento inicial si `overlap_words > 0`).
`word_count`	`int`	Total de palabras en `text` (`unique_words + overlap_words`).
`overlap_words`	`int`	Cantidad de palabras al inicio de `text` repetidas del chunk anterior de la misma sección.
`unique_words`	`int`	Palabras nuevas aportadas por este chunk. **Importante:** Para estadísticas de volumen léxico debe sumarse `unique_words`, nunca `word_count`.

Campo	Tipo	Descripción y Uso en LLMs / RAG
`page_start` / `page_end`	`int`	Rango de páginas PDF (índice base 1) donde aparece el texto **propio** (`unique_words`), excluyendo el solapamiento.
`page_label_start` / `page_label_end`	`string`	Numeración impresa definida en el catálogo del PDF (p. ej. `"iv"`, `"12"`). Permite al LLM citar tanto la página PDF como la página impresa.
`related_table_ids`	`list[str]`	Lista de `table_id` de tablas válidas ubicadas en las páginas cubiertas por el chunk. Permite inyectar tablas asociadas en el prompt del LLM.
`quality_score`	`float`	Promedio de `overall_quality_score` de las páginas del chunk (`0.0` a `1.0`).
`language`	`string`	Código ISO de idioma detectado por `langdetect` (`"es"`, `"en"`, `"unknown"`).
`text_sha256`	`string`	Hash SHA-256 del texto en minúsculas para deduplicación inter-documental.

3.2. `tables.jsonl` y `tables_rejected.jsonl` (Evidencia Tabular para LLMs)

Extrae y transforma tablas de PDF en tres representaciones complementarias:

- `table_id` (`tbl_<16hex>`), `document_id`, `page_number`, `strategy` (`"lines"` o `"text"`), `bbox`, `rows`, `columns`.
- `table_confidence`: `"high"` (`strategy == "lines"` y `filled_ratio >= 0.5`), `"medium"` (pasa `judge()`) o `"low"` (rechazada).
- `quality_features`: Diccionario con `filled_ratio`, `fragment_ratio` (proporción de celdas con `<= 2` caracteres), `lower_start_ratio` (proporción de celdas que inician en minúscula, indicador clave de prosa partida), `avg_words_per_cell` y `page_area_share`.

- `caption`: Título o leyenda detectada en la franja de 45 puntos por encima de la tabla (`Table/Tabla/Cuadro/Figura...`).
- `markdown`: Representación en tabla Markdown lista para inyectar directamente en el prompt de un LLM (hasta 40 filas).
- `headers` y `records`: Lista de diccionarios `{columna: valor}` ideal para análisis cuantitativo o carga en Pandas/SQL, más `matrix` (lista de listas).
- En `tables_rejected.jsonl`, se omite `records` y `markdown`, y se añade `reject_reason` explicando por qué se descartó la rejilla.

3.3. `sections.jsonl` (Estructura Jerárquica Documental)

Preserva las unidades lógicas completas del documento antes de la fragmentación:

- `section_id`, `document_id`, `title` (y su alias `section_title`), `title_source` (`"detected"` o `"none"`), `heading_confidence` (`0.0`–`1.0`), `heading_level` (`1`, `2`, `3` o `null`), `page_start`, `page_end`.
- `paragraphs`: Lista de objetos `{"page_number": int, "text": str, "block_id": int | null}` que conserva la procedencia exacta por bloque y página de cada párrafo.
- `text`, `word_count`, `paragraph_count`, `language`.

3.4. `pages.jsonl` (Auditoría Página por Página y Layout)

Contiene la evidencia cruda y procesada de cada página física:

- Identificación y dimensiones: `page_id` (`doc_<16hex>_P00001`), `document_id`, `filename`, `page_number`, `page_label`, `width`, `height`.
- Tres capas de texto: `native_text` (texto bruto de PyMuPDF), `ocr_text` (texto bruto de Tesseract si se aplicó OCR) y `clean_text` (texto final normalizado sin cabeceras ni pies repetitivos).
- Trazabilidad de limpieza y alertas: `removed_edge_lines` (líneas de cabecera/pie eliminadas) y `page_flags` (`"ocr"`, `"ocr_low_confidence"`, `"empty"`, `"low_text"`, `"no_layout_blocks"`, `"image_page"`, `"page_error"`).
- Métricas de calidad: `density_score`, `encoding_score`, `legibility_score`, `token_structure_score`, `text_quality_score`, `layout_score`, `ocr_score`, `structure_score`, `table_score`, `overall_quality_score` y `extraction_quality_score`.
- Geometría: `blocks` (con `bbox`, `spans` tipográficos y señales de encabezado) e `images_metadata`.

3.5. `documents.jsonl` (Catálogo Documental)

Resumen técnico y bibliográfico por PDF:

- Huella e inventario: `document_id`, `filename`, `file_size_bytes`, `sha256`, `page_count`, `extracted_pages`, `encrypted`, `processed_at`.
- Metadatos internos vs. inferidos: `title`, `author`, `subject`, `creator`, `producer`, `pdf_creation_date` (extraídos del diccionario interno del PDF) frente a `inferred_title`, `title_source` (`"pdf_metadata"`, `"largest_font_cover"`, `"largest_font_pages_2_3"`, `"filename"`), `year_candidates` (hasta 3 años más frecuentes en páginas 1–2) y `document_language`.
- Estadísticas de extracción y cobertura: `profile_type`, `profile_type_observed`, `total_words`, `total_tables`, `tables_rejected`, `total_sections`, `sections_with_detected_title`, `total_chunks`, `ocr_pages`, `low_text_pages`, `headings_demoted`, `section_words`, `chunk_unique_words`, `chunk_coverage`, `average_quality`, `average_extraction_quality`, `repeated_header_footer_patterns`.

3.6. Archivos JSON de Control (`manifest.json`, `integrity_report.json`, `quality_report.json`, `data_dictionary.json`)

- **`integrity_report.json`**: Evalúa 9 controles automáticos y emite un estado global (`PASS`, `PASS_WITH_WARNINGS` o `FAIL`), además de listar en `review_pages` todas las páginas que requieren inspección visual humana antes de ser citadas.
- **`quality_report.json`**: Consolida métricas globales del corpus, distribución de calidad de páginas (`excellent_0.80_plus`, `good_0.60_0.80`, `fair_0.40_0.60`, `poor_below_0.40`) y el resumen por documento (`documents_detail`).
- **`manifest.json`**: Garantiza la reproducibilidad científica registrando versiones exactas de Python, librerías y Tesseract (`libraries`), hiperparámetros (`configuration`), conteos y hashes `sha256` de cada PDF de entrada (`inputs`) y de cada archivo generado (`outputs`).
- **`data_dictionary.json`**: Documenta semánticamente cada campo para evitar malas interpretaciones analíticas (p. ej., advertir que `year_candidates` o los metadatos internos del PDF no prueban la fecha real de publicación).

---

4. Parámetros de Configuración y Heurísticas Clave

Categoría	Parámetro	Valor por Defecto	Fu
Lotes	`MAX_DOCUMENTS`	`50` (None para todos)	Lí pr
OCR	`OCR_ENABLED` / `OCR_LANGUAGE` / `OCR_DPI`	`True` / `"spa+eng"` / `200`	Co Te
OCR	`OCR_MIN_CHARS` / `OCR_MIN_WORDS` / `OCR_MIN_NATIVE_QUALITY`	`80` / `20` / `0.35`	Ur di
Estructura	`HEADING_FONT_RATIO` / `HEADING_MIN_SCORE`	`1.18` / `2.0`	Pr fu la ac er
Estructura	`HEADING_REPEAT_MIN_PAGES` / `HEADING_REPEAT_MIN_SHARE`	`4` / `0.08`	M de er co
Bordes	`HEADER_FOOTER_THRESHOLD` / `HEADER_FOOTER_EDGE_LINES`	`0.35` / `3`	Fr nú su in ca
Chunking	`CHUNK_TARGET_WORDS` / `CHUNK_MAX_WORDS` / `CHUNK_MIN_WORDS`	`550` / `800` / `60`	Ta un se pr
Chunking	`CHUNK_OVERLAP_WORDS` / `CHUNK_COVERAGE_MIN`	`80` / `0.995`	Ve er se ex
Tablas	`TABLE_TEXT_MAX_LOWER_START` / `TABLE_TEXT_MAX_AREA`	`0.25` / `0.70`	En re ce

Categoría	Parámetro	Valor por Defecto	Fu
			oc pá
Índices	`EMBEDDING_MODEL`	`paraphrase-multilingual-MiniLM-L12-v2`	M di ín a

---

5. Guía de Uso para Análisis con LLMs y Sistemas RAG

5.1. Construcción del Contexto para un LLM (Patrón Recomendado)

Al recuperar fragmentos (vía `search_local` con TF-IDF, búsqueda semántica en `faiss.index` o consulta SQL en corpus.duckdb), se recomienda ensamblar cada bloque de evidencia para el prompt del LLM incluyendo sus metadatos de trazabilidad y las tablas vinculadas:

1. **Filtrado de calidad**: Excluir o marcar con advertencia los chunks con `quality_score < 0.40` o cuyas páginas aparezcan en `integrity_report.json -> review_pages`.
2. **Inyección de tablas**: Si `chunk["related_table_ids"]` no está vacío, consultar `tables.jsonl` (o la tabla `tables` en DuckDB) por `table_id` y anexar el campo `markdown` junto con su `caption`.
3. **Formato de bloque de evidencia para el Prompt**:
    - **ID de Evidencia**: `{chunk_id}`
    - **Documento**: `{filename}` (`{document_id}`)
    - **Sección**: `{section_title}` (Confiable: `{section_title_reliable}`)
    - **Ubicación**: Páginas PDF `{page_start}`–`{page_end}` (Etiqueta impresa: `{page_label_start}`–`{page_label_end}`)
    - **Texto**: `{text}`

5.2. Consultas Analíticas Directas con DuckDB

El archivo `corpus.duckdb` permite cruzar chunks, páginas, documentos y tablas mediante SQL estándar sin cargar los JSONL completos en memoria:

- Consultar fragmentos de alta calidad junto con el título inferido del documento y las tablas de la misma página.
- Auditar qué secciones fueron absorbidas revisando `merged_section_ids`.

- Contabilizar el volumen léxico real sumando `SUM(unique_words)` agrupado por `document_id`.

---

6. Revisión Técnica del Código (Auditoría de Ingeniería)

6.1. Fortalezas Principales del Diseño

1. **Resolución efectiva de pérdidas silenciosas**: La transición de V5.1 a V5.2 resuelve el problema crítico de los *pipelines* RAG tradicionales que descartan secciones cortas o contaminan los números de página por culpa de la ventana de solapamiento (`overlap`).
2. **Separación entre calidad técnica y estructura (`extraction_quality_score` vs. `overall_quality_score`)**: Evita penalizar injustamente portadas o páginas de cierre con poco texto cuando su codificación y legibilidad son óptimas.
3. **Degradación elegante (*Graceful Degradation*)**: Si fallan `pdfplumber`, `DuckDB`, `TF-IDF` o `SentenceTransformers/FAISS`, el pipeline captura la excepción en `errors.jsonl` y preserva intacto el corpus base (`.jsonl` y `.parquet`).
4. **Higiene tipográfica y heurística en `HeadingDetector` y `AdaptiveTableExtractor`**: El uso de señales negativas (paréntesis desbalanceados de notas al pie, proporción de dígitos en ejes de gráficos, razón de celdas que inician en minúscula) reduce drásticamente la fragmentación artificial del texto.

6.2. Hallazgos, Bugs Sutiles y Casos Borde Detectados

#	Severidad	Ubicación en el Código	De
1	Media-Alta	`infer_repeated_lines` (Línea 954) y `DocumentExtractor.extract` (Líneas 1168–1256)	O ca 1) in de es es ca cl de ca he y

#	Severidad	Ubicación en el Código	De
			de `cl` ur `is`
2	Media	`ChunkBuilder.build` (Líneas 1086–1095) vs. `CorpusBuilder.process` (Línea 1330)	**As pa** ur `ti` el de `ur` se pa `ch` `se` `ch` `s|` Es `ch` ar pe
3	Media	`ChunkBuilder.absorb` (Líneas 1072–1078 y 1090–1091)	**At fu** Cu ur co se m co `se` **ar** `se` tít sa `me`

#	Severidad	Ubicación en el Código	De
4	**Baja-Media**	`AdaptiveTableExtractor.extract` (Líneas 943–944)	**Co de** `st ar` de ac pá co bo qu `"t` nu
5	**Baja**	`CorpusBuilder` (Líneas 1296–1396)	**Co co** ac di lo co tip `bb` ex pu Co pa
6	**Baja (Higiene)**	Celdas de Colab 1–4, 8 y 9	**De có fin** `fa` ni 9 re qu es `Co t.`